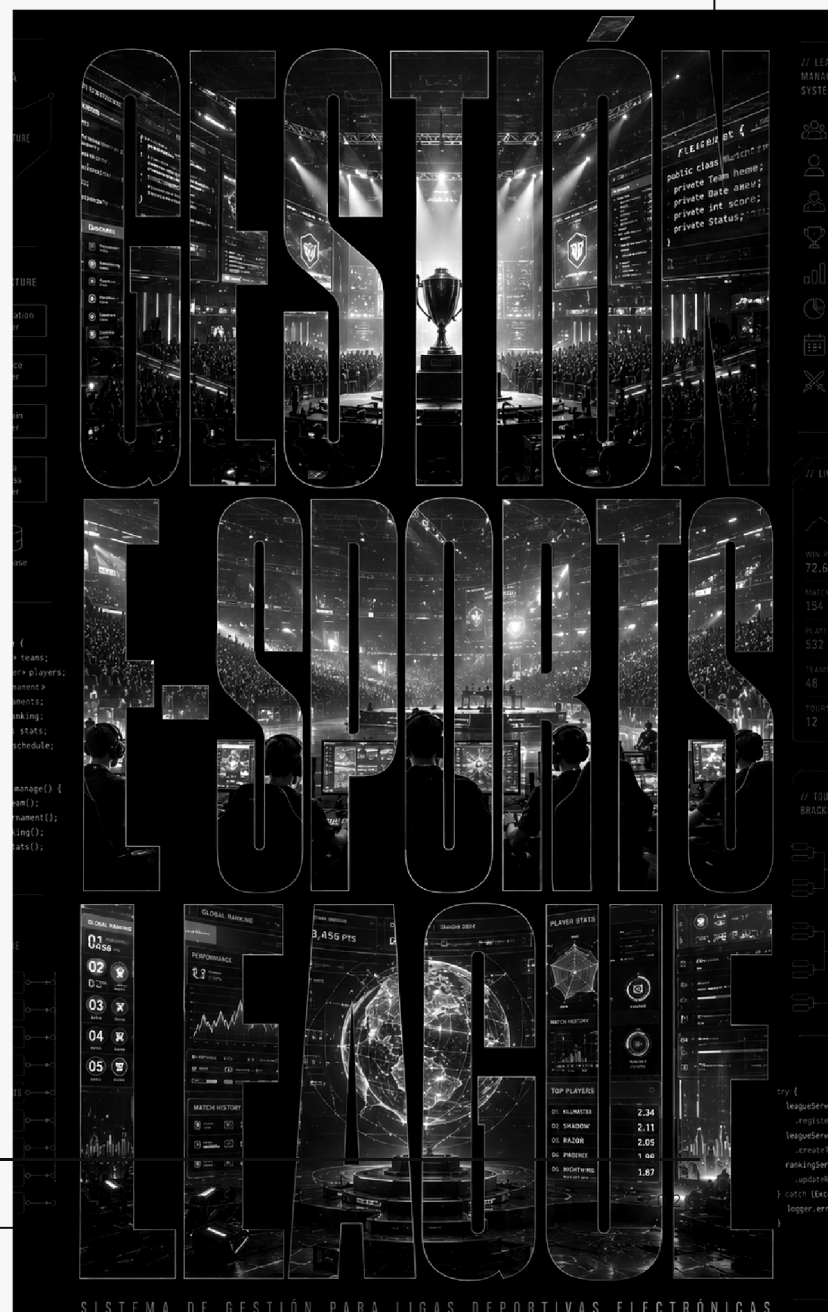


LIGA MEMORIA TÉCNICA

GESTIÓN DE UNA LIGA DE E-SPORTS ·
LIGA1GSY · IVÁN GUTIÉRREZ MATEOS

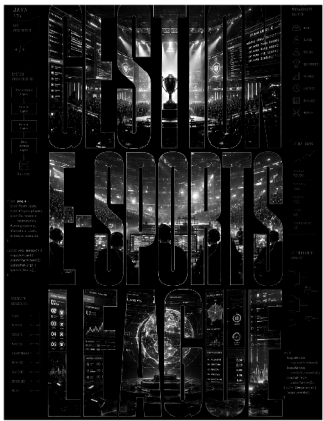
APLICACIÓN JAVA POR CONSOLA



Índice editorial

Memoria técnica del proyecto LIGA1GSY.

ALUMNO: IVÁN GUTIÉRREZ MATEOS
PROYECTO: LIGA1GSY
MEMORIA TÉCNICA - PROYECTO FINAL 1GSY



01	DESCRIPCIÓN GENERAL DEL PROYECTO
03	JUSTIFICACIÓN DE PROGRAMACIÓN ORIENTADA A OBJETOS
05	JUSTIFICACIÓN DE MATRICES
07	JUSTIFICACIÓN DE ESTRUCTURAS DINÁMICAS PARA DUPLICADOS
09	JUSTIFICACIÓN DE PILA
11	GESTIÓN DE EQUIPOS Y PLANTILLAS
13	REGISTRO DE PARTIDOS
15	FORMATO DEL TORNEO
17	FUNCIONAMIENTO GENERAL DEL MENÚ
19	REVISIÓN DE CONTENIDOS DEL TEMARIO

02	ESTRUCTURA DE CLASES
04	JUSTIFICACIÓN DE ARRAYS
06	JUSTIFICACIÓN DE LISTAS DINÁMICAS
08	JUSTIFICACIÓN DE COLA
10	GESTIÓN DE PERSONAS DE LA LIGA
12	CONVOCATORIAS Y SANCIONES
14	CLASIFICACIÓN Y ESTADÍSTICAS
16	EXPLICACIÓN DE EXCEPCIONES
18	FUNCIONALIDADES EXTRA AÑADIDAS
20	CONCLUSIÓN

DESCRIPCIÓN GENERAL DEL PROYECTO

El proyecto LIGA1GSY es una aplicación Java orientada a la gestión de una liga de e-sports. La idea principal es simular una competición organizada por 1GSY, donde existen personas de la liga, equipos, entrenadores, jugadores titulares y suplentes, partidos, calendario, incidencias, sanciones, clasificación, eventos y un historial de acciones.

La aplicación se ejecuta desde la clase MainEsports. Al arrancar, se crea una liga llamada "1GSY WORLD CHAMPIONSHIP SERIES", se cargan datos de prueba y se muestra un menú por consola. Ese menú permite probar las funcionalidades principales que pide el enunciado del PDF: gestión de personas, equipos, fichajes, calendario, cola de partidos, resultados, incidencias, sanciones, clasificación, estadísticas e historial.

El programa no concentra toda la lógica en el main. La clase MainEsports se encarga sobre todo de mostrar el menú, leer datos por teclado y llamar a los métodos correspondientes. La lógica principal está repartida entre Liga, Equipo, Partido, Jugador, Entrenador, Incidencia y las clases auxiliares.

MainEsports-

Es la clase principal del programa. Contiene el método main, crea el objeto Liga, carga datos iniciales y muestra el menú de consola. También incluye métodos auxiliares para leer enteros, decimales y textos, pedir roles, crear jugadores libres, gestionar fichajes, registrar incidencias, apostar o añadir apoyo económico al siguiente partido y disputar partidos.

Liga

Es la clase central del sistema. Guarda el nombre de la liga, las personas registradas, los equipos, las incidencias, los eventos, los partidos pendientes, el historial y el calendario. También controla duplicados mediante HashSet. Desde esta clase se crean personas, equipos y partidos, se muestra el calendario, se disputan partidos, se registra el historial, se muestran personas, equipos y clasificación, y se gestionan fichajes, transferencias, incidencias y sanciones.

PersonaLiga

Es una clase abstracta que representa una persona común dentro de la liga. Sus atributos son id, nombre, nickname, edad y salarioBase. Define los métodos abstractos calcularCosteMensual() y mostrarResumen(), porque un jugador y un entrenador no calculan su coste ni muestran su resumen exactamente de la misma forma. También incluye getters, setters con validación y toString().

Jugador

Es una subclase de PersonaLiga e implementa la interfaz Entrenable. Representa a un jugador de e-sports. Sus atributos propios son rol, nivelMecanico, nivelEstrategico, partidasJugadas, mvpTotales, sancionado, puntosMvp y highlightsRealizados. Calcula su rendimiento a partir de sus niveles y estadísticas, permite entrenar, sumar partidas, sumar MVP y registrar highlights.

El estado de sanción se consulta con el método esSancionado(). Este método devuelve true si el jugador está sancionado y false si puede participar.

Entrenador

Es una subclase de PersonaLiga. Representa al entrenador de un equipo. Sus atributos propios son experiencia, especialidad y victoriasTotales. Su coste mensual depende del salario base, la experiencia y las victorias acumuladas.

Entrenable

Es una interfaz implementada por Jugador. Define el comportamiento de una entidad entrenable mediante los métodos entrenar() y calcularRendimiento(). En este proyecto entrenar() mejora los niveles del jugador y calcularRendimiento() devuelve un valor numérico aproximado de su rendimiento.

Equipo

Representa un equipo de la liga. Tiene nombre, ciudad, entrenador, presupuesto, victorias, derrotas, puntos a favor, puntos en contra, un array fijo de titulares, una lista dinámica de suplentes y una lista de patrocinadores. Permite asignar entrenador, fichar titulares, fichar suplentes, eliminar jugadores, comprobar si un jugador pertenece al equipo, generar convocatorias válidas, calcular coste total, calcular valor de patrocinio, calcular rendimiento del equipo y mostrar la plantilla completa.

Partido

Representa un enfrentamiento entre dos equipos. Sus atributos principales son id, jornada, equipo local, equipo visitante, puntos local, puntos visitante, MVP y estado de disputado. También incluye apoyo económico o apuestas de comunidad y un highlight del partido. Valida que el partido no tenga equipos nulos ni un equipo contra sí mismo. Al disputarse, genera o registra un resultado, actualiza estadísticas de equipos y jugadores, elige MVP y marca el partido como disputado.

Incidencia

Representa una incidencia de la liga. Guarda id, tipo, descripción, equipo relacionado y jugador relacionado. Si la incidencia es de tipo SANCION o EXPULSION y tiene un jugador asociado, aplica la sanción marcando al jugador como sancionado.

Evento

Representa actividades del día del torneo, como apertura, música, comida, shows, premios o patrocinadores. Tiene hora, nombre, descripción, categoría y protagonista.

Patrocinador

Representa un patrocinador asociado a un equipo. Guarda nombre y aportación económica. El equipo usa esta información para calcular el valor total de patrocinadores.

Ro1

Es un enum con los roles obligatorios de jugadores: TOP, JUNGLE, MID, ADC y SUPPORT.

TipoIncidencia

Es un enum con tipos de incidencias: SANCION, EXPULSION, ERROR_TECNICO, PARTIDO_APLAZADO y OTRO.

CategoriaEvento

Es un enum adicional para clasificar eventos: MUSICA, COMIDA, SHOW, PREMIOS y PATROCINADOR.

Excepciones personalizadas

El proyecto incluye PersonaDuplicadaException, EquipoDuplicadoException, Ro1NoDisponibleException, PartidoInvalidoException y JugadorSancionadoException. Todas heredan de Exception y se lanzan cuando se incumplen reglas del sistema.

JUSTIFICACIÓN DE PROGRAMACIÓN ORIENTADA A OBJETOS

El proyecto usa encapsulación porque los atributos principales de las clases son privados y se accede a ellos mediante getters, setters y métodos propios.

Usa herencia porque Jugador y Entrenador heredan de PersonaLiga. Ambas clases comparten datos comunes, pero cada una tiene atributos y comportamientos específicos.

Usa clases abstractas porque PersonaLiga no representa una persona concreta que deba instanciarse directamente, sino una base común para jugadores y entrenadores.

Usa polimorfismo porque la lista de personas de Liga es `ArrayList<PersonaLiga>`, pero puede contener objetos Jugador y Entrenador. Cada subclase implementa de forma distinta `calcularCosteMensual()` y `mostrarResumen()`.

Usa interfaces porque Jugador implementa Entrenable, indicando que los jugadores pueden entrenar y calcular rendimiento.

Usa sobreescritura porque Jugador, Entrenador, Equipo, Partido, Evento, Patrocinador y PersonaLiga redefinen `toString()`, y Jugador y Entrenador implementan los métodos abstractos heredados.

JUSTIFICACIÓN DE ARRAYS

El uso obligatorio de arrays se cumple en Equipo con el atributo:

```
Jugador[] titulares = new Jugador[5];
```

Este array representa los cinco titulares del equipo. Tiene sentido real porque cada posición se relaciona con un rol mediante el ordinal del enum Rol. De esta forma, TOP, JUNGLE, MID, ADC y SUPPORT tienen una posición concreta y se evita que haya dos titulares con el mismo rol.

También se usa un array en `generarConvocatoriaValida()`, donde se devuelve un `Jugador[]` de exactamente 5 jugadores para disputar un partido. Esta convocatoria comprueba que los jugadores no estén sancionados y que, si falta un titular o está sancionado, pueda entrar un suplente compatible.

JUSTIFICACIÓN DE MATRICES

El calendario usa una matriz bidimensional:

```
Partido[][] calendario = new Partido[10][5];
```

Cada fila representa una jornada y cada columna representa un hueco de partido dentro de esa jornada. Cuando se crea un partido, Liga busca la primera columna libre de la jornada y guarda el partido en esa posición.

La matriz no es decorativa: se usa para almacenar partidos, buscar huecos disponibles, mostrar el calendario completo y consultar una jornada concreta.

APARTADO

006

JUSTIFICACIÓN DE LISTAS DINÁMICAS

El proyecto usa ArrayList en varias partes:

```
ArrayList<PersonaLiga> personas: guarda jugadores y entrenadores registrados.
```

```
ArrayList<Equipo> equipos: guarda todos los equipos de la liga.
```

```
ArrayList<Incidencia> incidencias: guarda las incidencias registradas.
```

```
ArrayList<Jugador> suplentes: guarda los suplentes de cada equipo.
```

```
ArrayList<Patrocinador> patrocinadores: guarda patrocinadores de cada equipo.
```

```
ArrayList<Evento> eventosDelDia: guarda el horario de eventos del torneo.
```

La lista de suplentes cumple una función importante porque permite añadir, eliminar, recorrer y buscar suplentes compatibles para sustituir titulares sancionados o ausentes.

APARTADO

007

JUSTIFICACIÓN DE ESTRUCTURAS DINÁMICAS PARA DUPLICADOS

La clase Liga usa HashSet para controlar duplicados:

```
HashSet<String> idsPersonas: evita registrar dos personas con el mismo ID.
```

```
HashSet<String> nombresEquipos: evita crear dos equipos con el mismo nombre.
```

```
HashSet<String> idsPartidos: evita crear dos partidos con el mismo ID.
```

Se normalizan las claves con `trim()` y `toLowerCase()`, por lo que se detectan duplicados aunque haya diferencias de mayúsculas, minúsculas o espacios al principio y al final.

APARTADO

008

JUSTIFICACIÓN DE COLA

La clase Liga usa una cola para los partidos pendientes:

```
Queue<Partido> partidosPendientes = new LinkedList<Partido>();
```

NOTA TÉCNICA

Los partidos se añaden con `add()` al crearlos. El siguiente partido se consulta con `peek()` y se disputa siguiendo el orden de llegada. Cuando se disputa, se elimina con `poll()`. Esto refleja el comportamiento FIFO: el primer partido que entra es el primero que se juega. **IMPORTANTE:** `poll()` es como el `remove()`, pero lo uso porque devuelve null y así no tengo que manejar errores en cada caso.

El menú permite ver el siguiente partido pendiente, mostrar todos los pendientes, disputar el siguiente partido y vaciar la cola cuando proceda.

APARTADO

009

JUSTIFICACIÓN DE PILA

La clase Liga usa una pila para el historial de acciones:

```
Stack<String> historialAcciones;
```

Las acciones se guardan con `push()`. La última acción se consulta como la más reciente y se puede eliminar con `pop()` en `deshacerUltimaAccion()`. Esto refleja el comportamiento LIFO: la última acción registrada es la primera que se deshace o elimina del historial.

Además, existe una segunda pila llamada `accionesDeshacer`, que guarda información interna para poder deshacer algunas acciones reales, como altas de personas, creación de equipos o creación de partidos pendientes.

APARTADO

010

GESTIÓN DE PERSONAS DE LA LIGA

El sistema permite dar de alta personas, listar todas las personas registradas, buscar una persona por identificador, modificar datos generales y eliminar personas cuando procede.

La eliminación tiene validaciones. Por ejemplo, no se debe eliminar un jugador que pertenece a un equipo ni un entrenador que está asignado a un equipo. Esto evita dejar referencias incorrectas dentro de las plantillas.

APARTADO

011

GESTIÓN DE EQUIPOS Y PLANTILLAS

Cada equipo tiene nombre, ciudad, entrenador, presupuesto, victorias, derrotas, puntos a favor, puntos en contra, titulares y suplentes.

El sistema permite crear equipos, asignar entrenadores, fichar jugadores como titulares o suplentes, mostrar la plantilla completa, sustituir titulares por suplentes, eliminar jugadores y mostrar el coste total del equipo.

La regla de que no puede haber dos titulares con el mismo rol se controla usando el array de titulares. La posición del array depende del rol del jugador. Si ya existe un titular en esa posición, se lanza `RolNoDisponibleException`.

CONVOCATORIAS Y SANCIONES

Para disputar un partido, cada equipo genera una convocatoria válida mediante `generarConvocatoriaValida()`. Esta convocatoria debe tener exactamente 5 jugadores, uno por cada rol.

Si el titular existe y no está sancionado, juega el titular. Si el titular falta o está sancionado, el sistema busca un suplente compatible del mismo rol. Si no hay suplente válido, se lanza una excepción.

La sanción de un jugador se guarda en el atributo boolean `sancionado` de `Jugador`. El método `esSancionado()` permite consultar ese estado. Las clases `Equipo` y `Liga` usan ese método para impedir que un jugador sancionado entre en una convocatoria válida o sea elegido como candidato para determinadas operaciones.

Las incidencias de tipo `SANCION` o `EXPULSION` llaman a `aplicarSancion()`, que marca al jugador relacionado como sancionado.

REGISTRO DE PARTIDOS

La clase `Partido` contiene la información principal del enfrentamiento: identificador, jornada, equipo local, equipo visitante, puntos de cada equipo, MVP y estado de disputado.

El sistema permite crear partidos, registrar resultados manualmente, disputar partidos de forma automática, calcular el ganador, actualizar estadísticas de equipos, actualizar estadísticas individuales y evitar que se registre dos veces el mismo partido.

También se validan partidos no válidos, como partidos con equipos nulos o un equipo contra sí mismo.

CLASIFICACIÓN Y ESTADÍSTICAS

La clasificación se calcula en `Liga.obtenerClasificacionOrdenada()`. Se crea una copia de la lista de equipos y se ordena con `Collections.sort` y `Comparator`.

El criterio de ordenación es:

- Primero, más victorias.
- Segundo, mayor diferencia de puntos.
- Tercero, nombre del equipo en orden alfabético.

Las estadísticas de equipos se actualizan al disputar partidos: victorias, derrotas, puntos a favor y puntos en contra. Las estadísticas de jugadores también se actualizan: partidas jugadas, MVP, puntos MVP y highlights.

FORMATO DEL TORNEO

El proyecto incluye un formato competitivo ampliado. Primero se disputan tres jornadas iniciales con los seis equipos cargados en el calendario. Al terminar esas jornadas, la liga ordena a los equipos por puntos de clasificación, diferencia de puntos y nombre.

Los puntos de clasificación se calculan como 3 puntos por victoria. Solo pasan los cuatro primeros equipos. Los equipos en posición 5 y 6 quedan eliminados del torneo.

Después se genera una fase final al mejor de 3. El primer clasificado juega contra el segundo por el primer y segundo puesto. El tercer clasificado juega contra el cuarto por el tercer y cuarto puesto. Cada serie se va generando poco a poco: si un equipo gana dos partidos, la serie termina y no hace falta jugar un tercer partido.

Al finalizar las dos series se muestran los puestos finales del torneo.

EXPLICACIÓN DE EXCEPCIONES

PersonaDuplicadaException

Se lanza cuando se intenta dar de alta una persona con un ID que ya existe. Se usa en `Liga.altaPersona()`.

EquipoDuplicadoException

Se lanza cuando se intenta crear un equipo con un nombre ya registrado. Se usa en `Liga.crearEquipo()`.

RolNoDisponibleException

Se lanza cuando se intenta poner como titular a un jugador cuyo rol ya está ocupado, cuando un suplente no tiene el rol adecuado o cuando falta un jugador para completar una convocatoria válida.

PartidoInvalidoException

Se lanza cuando un partido tiene datos inválidos: ID vacío, jornada inválida, partido duplicado, equipos nulos, un equipo contra sí mismo, resultado negativo, empate o intento de registrar dos veces el mismo partido.

JugadorSancionadoException

Se lanza cuando un jugador sancionado no puede participar y no existe un suplente válido disponible para cubrir su rol.

Además, se usan excepciones estándar como `IllegalArgumentException` para validar datos incorrectos y `NumberFormatException` para controlar entradas numéricas inválidas por teclado.

FUNCIONAMIENTO GENERAL DEL MENÚ

El menú de `MainEsports` permite:

- Mostrar personas registradas.
- Buscar una persona por ID.
- Modificar nombre, nickname, edad y salario base de una persona.
- Eliminar personas cuando no pertenecen a un equipo.
- Mostrar jugadores.
- Mostrar equipos.
- Mostrar plantillas completas.
- Crear jugadores libres.
- Gestionar fichajes y transferencias.
- Sustituir un titular por un suplente compatible.

- Mostrar calendario.
- Consultar una jornada concreta.
- Ver el siguiente partido pendiente.
- Mostrar todos los partidos pendientes.
- Vaciar la cola de partidos pendientes.
- Añadir apoyo económico al siguiente partido.
- Registrar manualmente el resultado del siguiente partido.
- Disputar el siguiente partido automáticamente.
- Registrar, listar y buscar incidencias por equipo o jugador.
- Mostrar clasificación.
- Mostrar eventos del torneo.
- Mostrar historial.
- Deshacer la última acción del historial.
- Entregar premios finales.
- Salir.

APARTADO

018

FUNCIONALIDADES EXTRA AÑADIDAS

El proyecto incluye algunas ampliaciones coherentes con la temática:

Eventos del día del torneo.

Patrocinadores de equipos.

Apoyo económico o apuestas de comunidad antes de los partidos.

Highlights y premios finales.

Generación aleatoria de resultados y eventos.

Fase final al mejor de 3.

Estas funcionalidades no aparecen como obligatorias en el enunciado, pero son coherentes con una competición de e-sports y amplían el proyecto sin sustituir las partes obligatorias.

APARTADO

019

REVISIÓN DE CONTENIDOS DEL TEMARIO

El proyecto utiliza contenidos trabajados durante el curso:

CLASES Y OBJETOS.	CONSTRUCTORES.	GETTERS Y SETTERS.	ENCAPSULACIÓN.	HERENCIA.		
POLIMORFISMO.	CLASES ABSTRACTAS.	INTERFACES.	ENUMERADOS.	ARRAYS.	MATRICES.	
ARRAYLIST.	LINKEDLIST.	HASHSET.	QUEUE.	STACK.	COLLECTIONS.SORT.	COMPARATOR.
EXCEPCIONES PROPIAS.	TRY-CATCH.	SCANNER.	TOSTRING.	INSTANCEOF.		

También aparecen elementos adicionales como `java.util.Random`, patrocinadores, eventos, apoyo económico, highlights y premios.

APARTADO

020

CONCLUSIÓN

El proyecto está bien orientado al enunciado del PDF. Usa programación orientada a objetos, herencia, clase abstracta, interfaz, enums, arrays, matriz, ArrayList, HashSet, Queue, Stack, excepciones propias y menú por consola.

La estructura general separa la lógica del menú y reparte responsabilidades entre clases. Liga coordina el funcionamiento general, Equipo gestiona plantillas, Partido controla enfrentamientos y estadísticas, Jugador y Entrenador representan personas concretas de la liga, e Incidencia permite registrar sanciones y problemas durante la competición.

Los extras como eventos, patrocinadores, apuestas, highlights, premios y fase final aportan personalidad al proyecto, pero las funcionalidades obligatorias del enunciado también están presentes y se pueden comprobar desde el menú principal.